# What Improves Developer Productivity at Google? Code Quality

Nikhil Anand

March 2, 2026

This paper investigates what causes improvements in individual developer productivity at Google. Prior research mostly identified correlations (e.g., good tools correlate with productivity) or used tightly controlled experiments. The authors aim to go further by using panel data analysis, which allows stronger causal inference in a real-world setting. They analyze data from over 2,000 Google engineers using a longitudinal internal survey measuring self-reported productivity and workplace factors and detailed engineering logs (e.g., coding time, build times, code reviews).

Among these, satisfaction with project code quality emerges as the strongest factor. A follow-up lagged panel analysis shows that improvements in perceived code quality are followed by measurable improvements in objective productivity metrics (like reduced coding and deployment time), but not the other way around. This strengthens the claim that better code quality leads to higher developer productivity, rather than productivity simply enabling better quality.

## 1 Motivation

The motivation behind this paper comes from a very practical and high-stakes question: organizations invest heavily in tools, processes, and structural changes to improve developer productivity — but which of these actually cause productivity improvements in real-world settings? Prior research leaves leaders in an uncomfortable middle ground. On one side, controlled experiments (like testing a new debugging tool) can establish strong causality, but they often use simplified tasks, small samples, or student participants, raising doubts about real-world applicability. On the other side, large field studies in companies observe real developers doing real work, but they typically rely on cross-sectional correlations, which cannot rule out confounding factors. This gap creates uncertainty for decision-makers: should they invest in tooling, reduce meetings, reorganize teams, focus on code quality, or something else? The authors are motivated to bridge this methodological gap by applying panel data techniques in a real industrial context (Google), allowing them to make stronger causal claims without sacrificing ecological validity.

Specifically, the paper is motivated by:

- The need to move beyond simple correlations and identify what causes productivity improvements in practice.
- The limitations of prior research methods (controlled experiments vs. field studies) in addressing this question.
- The weaknesses of cross-sectional surveys, which cannot eliminate confounding factors (e.g., education, seasonal effects, personality traits).
- The desire to provide actionable, evidence-based guidance for engineering organizations.
- The broader research challenge of combining real-world data with stronger causal inference methods.

At its core, the paper aims to answer a single driving question: What truly improves developer productivity in practice, in a real organizational setting?

### 1.1 Previous (or Related) Work

A wide spectrum of prior research provides some answers to this question, but with significant caveats.

1. Ko and Myers' controlled experiment showed that a novel debugging tool called Whyline[1] helped Java developers fix bugs twice as fast as those using traditional debugging techniques.
2. At the other end of the spectrum, Murphy-Hill[2] and colleagues surveyed developers across three companies, finding that job enthusiasm consistently correlated with high self-rated productivity

While this evidence is compelling, organizational leaders are faced with many open questions about applying these findings in practice, such as whether the debugging tasks performed in that study are representative of the debugging tasks performed by developers in their organizations.

On one hand, software engineering research that uses controlled experiments can help show with a high degree of certainty that some practices and tools increase productivity, yet such experiments are by definition highly controlled, leaving organizations to wonder whether the results obtained in the controlled environment will also apply in their more realistic, messy environment. On the other hand, research that uses field studies ś often with cross sectional data from surveys or telemetry ś can produce contextually valid observations about productivity, but drawing causal inferences from field studies that rival those drawn from experiments is challenging.

## 2 Methodology

The study adopts a quasi-experimental longitudinal field design to investigate causal factors influencing developer productivity at Google. Rather than conducting a controlled laboratory experiment, the researchers analyze real-world data collected over time from professional software engineers. The central methodological choice is the use of panel data analysis, which allows the authors to make stronger causal inferences than traditional cross-sectional studies while preserving ecological validity. By observing the same engineers at multiple points in time, the study is able to isolate changes within individuals and control for stable personal characteristics that might otherwise confound results.

### 2.1 Data Sources

**Engineering Satisfaction (EngSat) Survey** The research integrates two primary data sources: a longitudinal internal survey and engineering logs data. The survey, known as the Engineering Satisfaction (EngSat) survey, is administered company-wide every three months. Engineers report their experiences over the previous quarter, including perceptions of productivity, code quality, technical debt, tooling, communication, and organizational factors. Engineers with at least six months of tenure are eligible, and each engineer appears at most twice in the final panel dataset. After joining survey responses with behavioral data, the authors obtained complete panel data for 2,139 engineers.

**Engineering Logs** In addition to survey data, the study incorporates detailed logs from internal engineering systems. These logs include data from version control systems, code review platforms, build and test systems, and calendar systems. From these sources, the researchers extracted objective measures such as active coding time, wall-clock coding duration, number of changelists merged, build latency, review rounds, review wait times, and time spent in meetings. This integration of subjective and objective data strengthens the study's validity by allowing perceived productivity to be compared against measurable work behavior.

### 2.2 Variables

**Dependent Variable** The primary dependent variable is self-rated productivity. Engineers were asked to rate their overall productivity over the previous three months on a five-point Likert scale ranging from "Not at all productive" to "Extremely productive." Although subjective measures can raise concerns

[1] Whyline for Java - https://github.com/amyjko/whyline
[2] The previous required reading

about bias, the authors validated this measure by training a random forest classifier using objective productivity metrics. The model achieved high precision and recall, demonstrating a strong relationship between self-reported productivity and actual work patterns. This validation step supports the use of subjective productivity as a meaningful outcome variable.

**Independent Variables** The study initially considered 42 potential productivity factors derived from both survey responses and logs data. After conducting multicollinearity checks using Variance Inflation Factor (VIF) analysis, three highly correlated variables were removed, leaving 39 independent variables in the final model. These variables were grouped into six conceptual categories: (1) code quality and technical debt; (2) infrastructure tools and support; (3) team communication; (4) goals and priorities; (5) interruptions; and (6) organizational and process factors. Some variables were perceptual (e.g., satisfaction with code quality, perceived hindrance from technical debt), while others were behavioral (e.g., build times, code review rounds, meeting duration). This broad set of predictors allowed the researchers to evaluate multiple dimensions of the software development environment simultaneously.

## 2.3 Panel Data Model

To analyze the relationship between productivity and these factors, the researchers employed a fixed-effects panel regression model. The general form of the model includes individual-specific effects and time-specific effects. Individual fixed effects capture all stable characteristics of a developer—such as inherent ability, education, personality, or long-term role assignment—while time effects account for broader company-wide changes or seasonal influences.

$$y_{it} = \alpha_i + \lambda_t + \beta D_{it} + \epsilon_{it} \tag{1}$$

where $y_{it}$ is the self-rated productivity of engineer $i$ at time $t$, $\alpha_i$ represents the individual fixed effect, $\lambda_t$ captures time fixed effects, $D_{it}$ is a vector of the 39 independent variables for engineer $i$ at time $t$, $\beta$ is the vector of coefficients to be estimated, and $\epsilon_{it}$ is the error term.

The model was then differenced between time periods. By analyzing changes within each engineer rather than differences between engineers, the fixed-effects approach eliminates time-invariant confounding factors automatically. In other words, any characteristic that does not change over time (such as education or long-standing skill level) is removed from the equation. This methodological choice substantially strengthens causal inference compared to cross-sectional correlation analysis.

$$\Delta y_{it} = \Delta\lambda_t + \beta \Delta D_{it} + \Delta\epsilon_{it} \tag{2}$$

To estimate the model parameters, the authors used Feasible Generalized Least Squares (FGLS)[3]. This method addresses issues of heteroskedasticity and serial correlation in panel data, improving statistical efficiency and robustness. Most continuous predictors were log-transformed so that estimated coefficients could be interpreted as elasticities, meaning that percentage changes in predictors correspond to percentage changes in productivity.

## 2.4 Lagged Panel Analysis

While the standard panel model identifies factors that are causally linked to productivity, it does not establish the direction of causality. To address this limitation, the authors conducted a lagged panel analysis focusing specifically on the relationship between code quality and productivity.

In this analysis, they tested two competing hypotheses: first, that changes in code quality at time $T-1$ lead to changes in productivity at time $T$; and second, that changes in productivity at time $T-1$ lead to changes in code quality at time $T$. To avoid reliance on subjective measures in this phase, the researchers

[3]Feasible Generalized Least Squares (FGLS) is a two-step estimation technique used in regression analysis to produce efficient, consistent estimators when the error terms are heteroscedastic or serially correlated (autocorrelated) with an unknown covariance structure

used logs-based productivity metrics such as median active coding time and wall-clock coding duration. The lagged dataset included 3,389 engineers.

By examining whether earlier changes in one variable predict later changes in the other, the researchers were able to infer directional causality. This additional modeling step significantly strengthens the study's conclusions regarding the impact of code quality.

# 3 Key Findings

## 3.1 Code Quality and Technical Debt

**Project Code Quality** Satisfaction with project code quality had the largest effect size among all predictors. Developers who reported improvements in the quality of the codebase they directly worked in also reported meaningful increases in productivity. Interestingly, satisfaction with dependency code quality was not significant. This suggests that developers are most impacted by the quality of the code they directly modify rather than upstream dependencies.

**Technical Debt** Technical debt within projects was also significantly associated with productivity changes. Developers who perceived lower technical debt reported higher productivity. Technical debt in dependencies was significant but had a smaller effect. This reinforces the practical intuition that maintainability and structural quality of codebases affect development velocity.

## 3.2 Infrastructure Tools and Support

**Tool and Infrastructure Satisfaction** Engineers who reported higher satisfaction with infrastructure and tools experienced higher productivity. Perceived innovation in tooling was also positively associated with productivity. Interestingly, both extremes of tool choice (too many or too few options) were associated with lower productivity. Similarly, rapid or slow changes in the developer tool stack were linked to reduced productivity.

**Build and Test Latency** Long build times were negatively associated with productivity. Engineers who experienced slower build cycles or reported being hindered by build and test processes showed lower productivity. This highlights that system latency and tooling friction create measurable productivity drag.

## 3.3 Team Communication

Among communication variables, code review dynamics were significant. Developers who experienced more review rounds or reported slow review processes tended to report lower productivity. This suggests that review inefficiencies introduce coordination overhead and delay feature delivery. However, physical distance from managers or reviewers, as well as organizational distance, were not significant. This challenges assumptions from distributed work literature that geographic distance inherently reduces productivity.

## 3.4 Goals and Priorities

Shifting project priorities showed one of the strongest negative associations with productivity. Developers who reported frequent priority changes experienced lower productivity. This finding emphasizes the importance of stability in project direction. Interruptions caused by goal shifts appear to impose cognitive switching costs that reduce development efficiency.

## 3.5 Organizational and Process Factors

Several structural variables were significant. Developers who experienced team or organizational changes, complicated processes, or direct manager changes reported lower productivity. Although the magnitudes

were smaller than code quality effects, these findings indicate that structural instability creates measurable productivity impact.

### 3.6 Non-Significant Findings

Some expected productivity drivers were not statistically significant:

- Documentation support and documentation hindrance
- Meeting time (both median and total time spent in meetings)
- Physical and organizational distance

These results contradict common developer perceptions that meetings and documentation are primary productivity inhibitors. The panel analysis suggests these factors may correlate with productivity perceptions but are not causally linked when controlling for within-person changes.

### 3.7 Statistical Analysis Summary

The core model estimated was a fixed-effects panel regression examining within-developer changes over time:

$$\Delta y_{it} = \Delta\lambda_t + \beta\Delta D_{it} + \Delta\epsilon_{it} \tag{3}$$

where $\Delta y_{it}$ is the change in self-rated productivity for engineer $i$ at time $t$, $\Delta\lambda_t$ captures time fixed effects, $\Delta D_{it}$ is the vector of changes in the 39 independent variables, $\beta$ is the vector of coefficients, and $\Delta\epsilon_{it}$ is the error term. The model was estimated using Feasible Generalized Least Squares (FGLS) to address heteroskedasticity and serial correlation in the panel data. The adjusted $R^2$ of the model came out to be $R^2 = 0.1019$. This indicates that approximately 10% of within-person productivity variation was explained by the included predictors.

### 3.8 Lagged Model Analysis

The two lagged models tested the directional relationship between code quality and productivity.

$$\Delta\text{Productivity}_T = \alpha + \beta_1\Delta\text{Code Quality}_{T-1} + \epsilon \tag{4}$$

$$\Delta\text{Code Quality}_T = \alpha + \beta_2\Delta\text{Productivity}_{T-1} + \epsilon \tag{5}$$

Results supported the first model but not the second. A 100% increase in project code quality at time $T-1$ led to (1) a 10% reduction in median active coding time; (2) a 12% reduction in wall-clock coding time; and (3) a 22% reduction in deployment latency at time $T$. In contrast, changes in productivity at time $T-1$ did not predict changes in code quality at time $T$. This asymmetry provides strong evidence that improvements in code quality lead to productivity gains, rather than productivity improvements enabling better code quality.

$$\text{Code Quality}_{T-1} \rightarrow \text{Productivity}_T \tag{6}$$

## 4 Implications

**1. Strategic Implications for Engineering Leadership** The most important implication of this study is that code quality is not merely a long-term maintainability concern — it is a direct driver of short-term developer productivity. Organizations often treat quality work (refactoring, reducing technical debt, improving design) as secondary to feature delivery. However, this research provides causal evidence that improving code quality leads to measurable improvements in productivity.

For engineering leadership, this reframes quality initiatives from "maintenance overhead" to "productivity investment." Allocating time for refactoring, improving code readability, reducing architectural complexity, and systematically managing technical debt should be considered productivity-enhancing strategies rather than trade-offs against velocity. In practical terms, this suggests:

- Embedding code health goals into quarterly planning.

- Protecting refactoring time in sprint cycles.
- Evaluating teams not only on feature output but also on structural quality improvements.
- Treating technical debt reduction as a productivity accelerator.

**2. Tooling and Infrastructure Investment** The findings also show that satisfaction with tools and infrastructure has a strong causal relationship with productivity. This has important implications for internal platform teams and developer experience (DevEx) organizations. Organizations should (1) prioritize reducing build and test latency; (2) invest in tooling innovation that directly addresses developer pain points; and (3) carefully manage the balance between tool choice and standardization.

Importantly, this study suggests that infrastructure investments yield measurable productivity returns. Investments in faster builds, smoother deployment pipelines, and stable tooling ecosystems are not merely developer satisfaction initiatives — they directly influence performance.

**3. Stability of Goals and Organizational Structure** Another major implication concerns project stability. Shifting priorities and frequent reorganizations were associated with reduced productivity. This suggests that cognitive switching costs and structural instability impose real productivity penalties. Organizations should (1) minimize abrupt priority changes; (2) provide clearer long-term direction; (3) avoid unnecessary reorganizations; and (4) stabilize reporting structures where possible. This finding challenges the assumption that agility requires constant change. While adaptability is necessary, excessive strategic churn may undermine productivity at the individual level.

**4. Reconsidering Common Productivity Assumptions** One of the most striking implications comes from the non-significant findings. Documentation quality and meeting time were not found to be causally linked to productivity in this panel analysis. This does not mean these factors are unimportant, but it suggests:

- Meetings may not inherently reduce productivity when controlling for other variables.
- Documentation may have benefits (e.g., onboarding, inclusion, knowledge sharing) that do not directly manifest as short-term productivity gains.
- Some developer complaints may reflect perceived friction rather than measurable productivity loss.

For managers, this implies that decisions should rely on longitudinal evidence rather than intuition or one-time survey correlations.